Challenge Scenario

Our SSH server is exhibiting unusual behavior: library linking errors are appearing, and critical directories seem to be missing despite their confirmed existence.

This raises suspicion of a potential **userland rootkit** manipulating the system.

The objective is to investigate anomalies in the library loading process and filesystem to uncover any hidden manipulations.

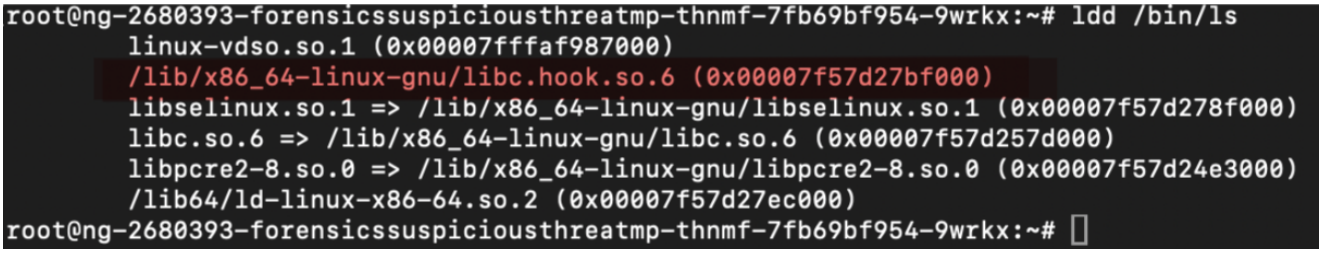
Initial Analysis

Upon gaining access to the target machine via SSH, I began by listing the directory contents to assess the system state.

However, the output appeared empty — highly unusual and immediately suggestive of tampering with standard system utilities.

To investigate further, I executed:

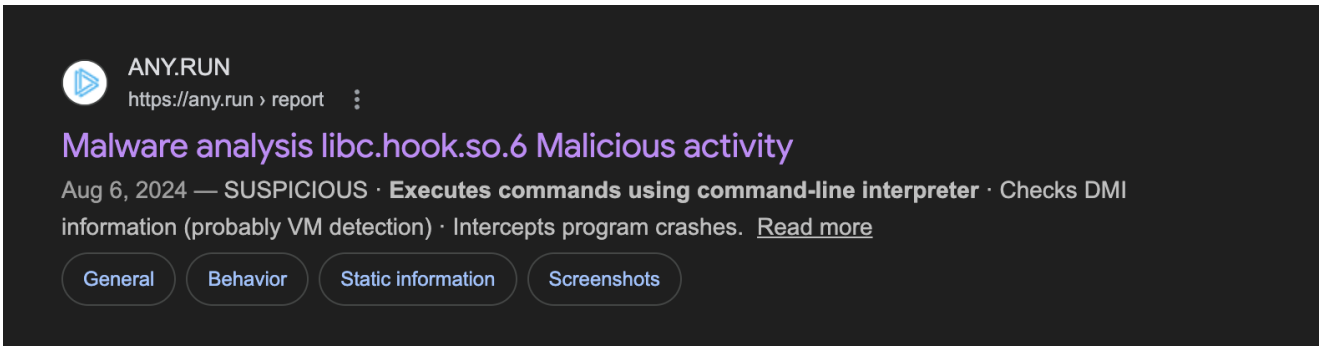
```
ldd /bin/ls
```

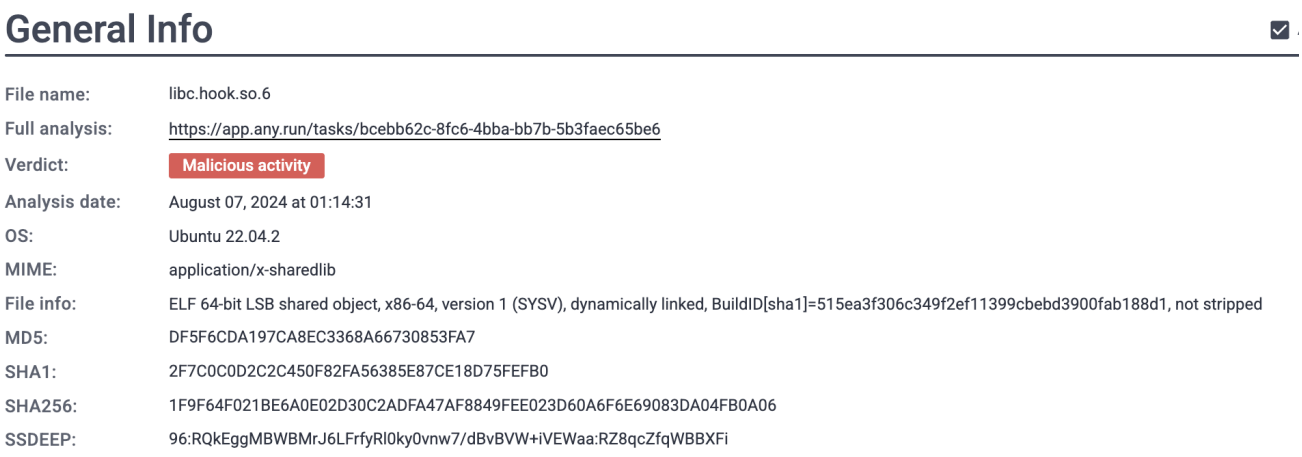


This revealed the shared libraries loaded by the `ls` binary. Among them, one entry stood out:

```
libc.hook.so.6
```

This library is highly suspicious. A quick search on google confirmed its association with malicious behavior, commonly used in **userland rootkits** to intercept and manipulate system calls.





General Info

File name:	libc.hook.so.6
Full analysis:	https://app.any.run/tasks/bcebb62c-8fc6-4bba-bb7b-5b3faec65be6
Verdict:	Malicious activity
Analysis date:	August 07, 2024 at 01:14:31
OS:	Ubuntu 22.04.2
MIME:	application/x-sharedlib
File info:	ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, BuildID[sha1]=515ea3f306c349f2ef11399cbebd3900fab188d1, not stripped
MD5:	DF5F6CDA197CA8EC3368A66730853FA7
SHA1:	2F7C0C0D2C2C450F82FA56385E87CE18D75FEFB0
SHA256:	1F9F64F021BE6A0E02D30C2ADFA47AF8849FEE023D60A6F6E69083DA04FB0A06
SSDEEP:	96:RQkEggMBWBMrJ6LFrfyRl0ky0vnw7/dBvBVW+iVEWaa:RZ8qcZfqWBBXFi

---

Malware Extraction

To analyze the suspicious library further, it needed to be extracted from the compromised environment.

Since direct file transfer was not available, I performed a manual extraction:

- Dumped the binary using a hexdump technique

```
root@ng-2680393-forensicssuspiciousthreatmp-ni1qf-584c94b7b7-qzt8c:/tmp# xxd -p
libc.hook.so.6
ERROR: ld.so: object '/lib/x86_64-linux-gnu/libc.hook.so.6' from /etc/ld.so.prel
oad cannot be preloaded (cannot open shared object file): ignored.
7f454c4602010100000000000000000003003e0001000000000000000000
0004000000000000004836000000000000000000400038000900400 0
1c001b0001000000040000000000000000000000000000000000000000
0000000000006806000000000006806000000000000010000000000000
0100000005000000010000000000000010000000000000010000000000
00001d0300000000001d0300000000000010000000000000010000000
0400000002000000000000002000000000000020000000000002401
0000000000024010000000000001000000000000010000000600000
e02d000000000000e03d000000000000e03d000000000004802000000000
0000680200000000000001000000000000020000006000000f02d0000
00000000f03d000000000000f03d000000000000c001000000000000c001
0000000000008000000000000004000000040000003802000000000000
3802000000000000380200000000002400000000000002400000000000
```

- Parse the binary in HexedIT
- Downloaded a clean 1:1 copy to my local machine

This allowed for safe offline analysis.

Rootkit Behavior

The extracted binary was loaded into a decompiler for inspection.

The analysis revealed that the shared object implements **function hooking** via `LD_PRELOAD` — a common technique employed by userland rootkits to intercept standard library calls.

File Hiding Mechanism

The rootkit intercepts directory listing functions and compares filenames against the following hardcoded strings: `ld.so.preload`
`pr3l04d_`

```
1  void* readdir64(int64_t arg1)
2  {
3      if (!*orig_readdir64)
4          *orig_readdir64 = dlsym(-1, "readdir64");
5
6      void* result;
7
8      while (true)
9      {
10         result = (*orig_readdir64)(arg1);
11
12         if (!result)
13             return nullptr;
14
15         if (strcmp(result + 0x13, "pr3l04d_") && strcmp(result +
               0x13, "ld.so.preload"))
16             break;
17     }
18
19     return result;
20 }
```

If a match is found, the corresponding file or directory is excluded from results, effectively hiding it from standard tools such as `ls`.

`fopen` Hooking

The rootkit also hooks the `fopen` function:

```
23  int64_t fopen(char* arg1, int64_t arg2)
24  {
25      if (!*orig_fopen)
26          *orig_fopen = dlsym(-1, "fopen");
27
28      if (!strstr(arg1, "ld.so.preload"))
29          return (*orig_fopen)(arg1, arg2);
30
31      *__errno_location() = 2;
32      return 0;
33  }
```

- It resolves the original `fopen` using `dlsym` to prevent recursion
- For most files, it behaves normally
- When attempting to open `ld.so.preload`, it returns `NULL`

This prevents detection of the preload configuration file, which is critical to maintaining the rootkit's persistence on the system.

Discovery

Based on the rootkit logic, I inferred that a hidden directory named `pr3l04d_` should exist on the filesystem.

By searching for this name explicitly, I was able to locate the hidden directory despite it not appearing in standard directory listings.

```
root@ng-2680393-forensicssuspicousthreatmp-ni1qf-584c94b7b7-qzt8c:~# find / -na
me "pr3l04d_"
ERROR: ld.so: object '/lib/x86_64-linux-gnu/libc.hook.so.6' from /etc/ld.so.prel
oad cannot be preloaded (cannot open shared object file): ignored.
/var/pr3l04d_
```

Flag Retrieval

Inside the `pr3l04d_` directory, I discovered a file named:

```
flag.txt
```

```
root@ng-2680393-forensicssuspiciousthreatmp-ni1qf-584c94b7b7-qzt8c:~# cd /var/pr
3l04d_
root@ng-2680393-forensicssuspiciousthreatmp-ni1qf-584c94b7b7-qzt8c:/var/pr3l04d_
# ls
ERROR: ld.so: object '/lib/x86_64-linux-gnu/libc.hook.so.6' from /etc/ld.so.prel
oad cannot be preloaded (cannot open shared object file): ignored.
flag.txt
root@ng-2680393-forensicssuspiciousthreatmp-ni1qf-584c94b7b7-qzt8c:/var/pr3l04d_
# cat flag.txt
ERROR: ld.so: object '/lib/x86_64-linux-gnu/libc.hook.so.6' from /etc/ld.so.prel
oad cannot be preloaded (cannot open shared object file): ignored.
HTB{Us3rL4nd_R00tK1t_R3m0v3dd!}
```

Reading the file revealed the final flag.